

## Analysis of main.py and generate.py

### Purpose of the Code:

#### 1. main.py

- Implements a reinforcement learning environment using the Double DQN algorithm.
- Simulates a competitive scenario between two agents (Red and Blue).
- Tracks metrics such as wins, losses, and ties.

#### 2. generate.py

- Generates initial conditions for the simulation.
- Randomly places two agents (Red and Blue) in a 3D space with specific physical parameters like position, orientation (Euler angles), and Mach numbers.
- Saves this data to a JSON file for later use.

---

### Details and Functionalities:

#### main.py

#### Core Components:

##### 1. Environment Setup:

- Uses a class Env to create the simulation environment.
- Resets the environment at the start of each episode.

## 2. Agent Training:

- Trains the Double DQN agent using state transitions (s, a, s', r, done) from the environment.
- Stores the neural network weights periodically for future use.

## 3. Performance Metrics:

- Tracks and records the number of wins, losses, and ties.
- Saves this data to a file for post-simulation analysis.

## 4. Simulation Loop:

- Runs up to 50,000 training iterations.
- Simulates episodes of up to 300 steps each, where agents interact with the environment.
- Decision-making is based on the agent's policy.

## 5. Outputs:

- Performance metrics.
- Trained neural network weights.

## Physical Parameters:

- env.red\_blood and env.blue\_blood: Represent health or a comparable survival metric.
- Reward Mechanism: Provides feedback for the agent's actions.
- Action Space: Encodes possible movements or strategies (e.g., a == 4 for a specific maneuver).

---

generate.py

## Core Components:

### 1. Random Initialization:

- Places two agents (Red and Blue) at random positions within a 3D space.
- Assigns random orientations (Euler angles) and speeds (Mach numbers).

### 2. Constraints:

- Ensures agents start with a minimum separation of 500 units.

### 3. Output:

- Saves the generated data to test\_record.json for use in simulations.

### Physical Parameters:

- Position: 3D coordinates (x, y, z) within specified bounds.
- Orientation: Euler angles indicating the agents' facing directions.
- Mach Number: Speed of the agents relative to the speed of sound, with randomized values between 0.3 and 0.9.

---

### Summary of Outputs:

#### 1. From main.py:

- Trained Agent: Double DQN policy.
- Performance Metrics: Win-loss-tie statistics.
- Saved Weights: Neural network checkpoints.

#### 2. From generate.py:

- Initial Conditions File: test\_record.json containing scenarios for simulations.

### Physical Meaning of Parameters:

- Position: Defines the spatial setup for the agents.
- Orientation: Determines the facing direction, potentially influencing movement or targeting.
- Mach Number: Reflects the agents' velocity, affecting interactions and outcomes.
- Health (Blood): Indicates survival capability.
- Reward: Guides learning by incentivizing desired outcomes.

The combination of these codes allows for simulating, training, and analyzing agent behaviors in a controlled competitive environment.